

Presentation Delivery Plan

Experimental Analysis of Sorting Algorithms

Maria Maioreescu

June 2, 2026

Audience Analysis

The presentation is addressed to:

university students,
highschool students,
software developers,
professors from computer science.

The presentation focuses mainly on practical benchmarking and experimental behaviour.

Presentation Structure

Slide 1 – Title and Introduction

Time: 30 seconds

Presentation Phase: Motivation

Operational Goal: Introduce the topic and research objective.

Presentation Method: Short oral introduction.

What I Say:

Today I will present my research paper called *Experimental Analysis of Sorting Algorithms*.

The goal of this project was to compare six classical sorting algorithms using different array sizes and input structures.

The project combines theoretical complexity analysis with practical benchmarking implemented in C.

Slide 2 – Importance of Sorting

Time: 40 seconds

Presentation Phase: Motivation

Operational Goal: Explain why sorting algorithms are important.

Presentation Method: Explanation with examples.

What I Say:

Sorting algorithms are fundamental in computer science and appear in databases, search engines, and operating systems.

Although theory focuses mainly on asymptotic complexity, practical performance also depends on implementation details and input structure.

Because of this, practical benchmarking is important in addition to theoretical analysis.

Slide 3 – Related Work

Time: 40 seconds

Presentation Phase: Association

Operational Goal: Connect the project with previous studies.

Presentation Method: Short overview.

What I Say:

This project was inspired by classical studies from Knuth and Cormen, as well as more recent work related to TimSort and adaptive sorting.

Unlike many previous studies, this project compares six algorithms using identical testing conditions and multiple input structures.

Slide 4 – Algorithms Used

Time: 1 minute

Presentation Phase: Learning

Operational Goal: Present the tested algorithms.

Presentation Method: Technical explanation.

What I Say:

The project compares Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort.

The quadratic algorithms become inefficient on large arrays, while Merge Sort and Quick Sort scale much better.

Radix Sort achieved the fastest performance in most large tests.

Slide 5 – Benchmark Methodology

Time: 1 minute 20 seconds

Presentation Phase: Learning

Operational Goal: Explain the benchmarking system.

Presentation Method: Step-by-step explanation.

What I Say:

The benchmarking framework was implemented entirely in C.

I tested random arrays, reversed arrays, and partially sorted arrays.

Execution time was measured using the `clock()` function, and small-array tests were averaged over multiple runs to reduce noise.

Slide 6 – Experimental Setup

Time: 45 seconds

Presentation Phase: Learning

Operational Goal: Present the testing environment.

Presentation Method: Technical overview.

What I Say:

The experiments were executed using GCC with optimization level O2.

The tested array sizes ranged from 10 elements up to one million elements.

Different input structures were used to observe how performance changes as the input size grows.

Slide 7 – Results for Small and Medium Arrays

Time: 1 minute

Presentation Phase: Learning

Operational Goal: Interpret the behaviour of algorithms at smaller scales.

Presentation Method: Comparative analysis.

Presentation Support: Results table and chart.

What I Say:

For small arrays, the performance differences between algorithms are relatively small. However, Radix Sort and Merge Sort already perform very efficiently even at medium sizes.

Insertion Sort also becomes surprisingly competitive when the input is almost sorted.

As the input size grows, the quadratic algorithms begin to slow down significantly.

Slide 8 – Results for Large Arrays

Time: 1 minute 20 seconds

Presentation Phase: Learning

Operational Goal: Explain performance differences on large datasets.

Presentation Method: Comparative analysis.

Presentation Support: Results table and chart.

What I Say:

For large arrays, the differences become extremely large.

Bubble Sort and Selection Sort become impractical because of their quadratic complexity.

Merge Sort remains stable and predictable, while Radix Sort becomes the fastest algorithm for large datasets.

Quick Sort performs very efficiently on random data, but some structured inputs still produce slowdowns.

Slide 9 – Analysis and Problems Encountered

Time: 1 minute 20 seconds

Presentation Phase: Association

Operational Goal: Discuss important practical observations.

Presentation Method: Discussion.

What I Say:

One important observation was the behaviour of Insertion Sort on nearly sorted input, where it became significantly faster.

Another important result involved Quick Sort on structured inputs.

Initially, the recursive implementation caused a stack overflow because of deep recursion.

After introducing a median-of-three pivot strategy, the algorithm became much more stable.

This shows that practical implementations may behave differently from theoretical expectations.

Slide 10 – Conclusion and Future Work

Time: 1 minute

Presentation Phase: Conclusion

Operational Goal: Summarize the study and future directions.

Presentation Method: Final summary.

What I Say:

In conclusion, this project compared six classical sorting algorithms across multiple array sizes and input structures.

The experiments confirmed the limitations of quadratic algorithms and highlighted the efficiency of Merge Sort and Radix Sort.

For future work, I would like to test additional data types and compare these algorithms with standard library implementations.

Thank you for your attention!